

[3조] 모터 제어 상위설계서

신유지, 이호윤, 이양배, 한소영

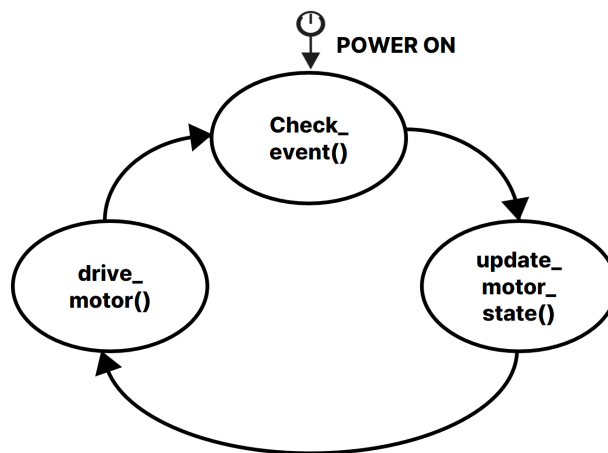
요구 사항

- 모터는 버튼과 UART 명령 두 가지 방식으로 제어할 수 있어야 한다.
- 시스템 시작 시 모터는 정지 상태여야 한다.
- 메인 루프는 UART 처리, 버튼 처리, 모터 상태 처리를 반복 수행한다.
- 방향 전환시 모터 부하 방지를 위해 전환 간 대기시간을 둔다.
- UART PWM duty rate 50 ~ 95% 이다.
- 위 요구사항에 명시되지 않은 세부 구현 방식은 자유롭게 설계한다.

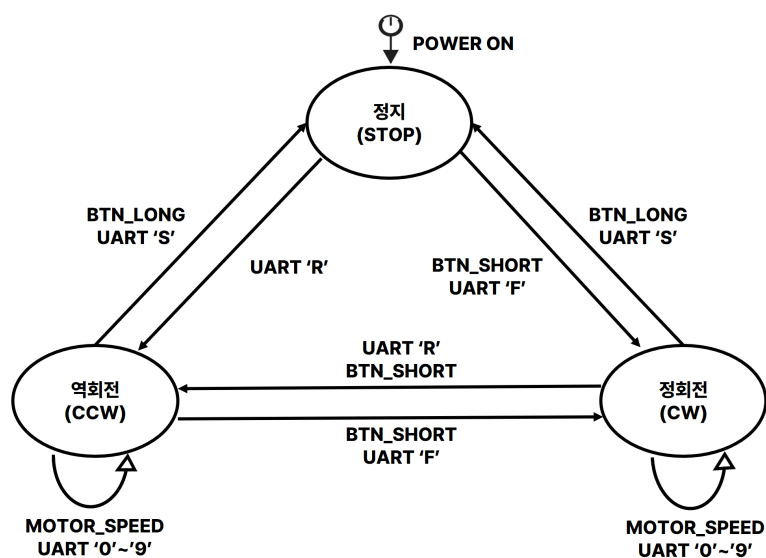
1. STATE

1-1. FSM

다이어그램 1. 메인 동작 흐름



다이어그램 2. 모터 상태 전이



1-2. MOTOR STATE

이름	값	설명
STOP	0	모터 정지
CW	1	모터 정회전
CCW	2	모터 역회전

1-3. MOTOR SPEED

이름	값	설명
0	50	PWM Duty 50% 설정
1	55	PWM Duty 55% 설정
2	60	PWM Duty 60% 설정
3	65	PWM Duty 65% 설정
4	70	PWM Duty 70% 설정
5	75	PWM Duty 75% 설정
6	80	PWM Duty 80% 설정
7	85	PWM Duty 85% 설정
8	90	PWM Duty 90% 설정
9	95	PWM Duty 95% 설정

2. EVENT (BUTTON/UART)

2-1. BUTTON/UART 이벤트

이름	값	설명
EVT_NONE	0	이벤트 없음 / 이벤트 소비 완료
EVT_BTN_SHORT	1	버튼 짧게 눌림 (버튼 눌린 시점부터 떴을 시간까지 측정. 눌린 시간 < 3000ms)
EVT_BTN_LONG	2	버튼 길게 눌림 (버튼이 눌린 채로 경과 시간 >= 3000ms)
EVT_UART	3	UART로 문자 수신 (수신 문자는 ISR에서 Uart_Data_In에 저장)

2-2. 상태별 이벤트 처리표

현재 상태	입력 이벤트/명령	다음 상태	수행 함수	전환 전 정지 지연
MOTOR_STOP	버튼 짧게 누름	MOTOR_CW	motor_cw()	없음
MOTOR_STOP	버튼 길게 누름	MOTOR_STOP	없음	없음

현재 상태	입력 이벤트/명령	다음 상태	수행 함수	전환 전 정지 지연
MOTOR_CW	버튼 짧게 누름	MOTOR_CCW	motor_stop_delay() → motor_ccw()	적용
MOTOR_CW	버튼 길게 누름	MOTOR_STOP	motor_stop()	없음
MOTOR_CCW	버튼 짧게 누름	MOTOR_CW	motor_stop_delay() → motor_cw()	적용
MOTOR_CCW	버튼 길게 누름	MOTOR_STOP	motor_stop()	없음
모든 상태	UART 'f'	MOTOR_CW	motor_cw()	미적용
모든 상태	UART 's'	MOTOR_STOP	motor_stop()	없음
모든 상태	UART 'r'	MOTOR_CCW	motor_ccw()	미적용
모든 상태	UART '0'~'9'	현재 상태 유지	TIM5_Out_PWM_Generation()	해당 없음

3. PWM 및 타이머

3-1. TIM2 모터 PWM

항목	설정값
TIM5 기준 주파수	96 MHz
PWM 주파수	1 kHz
출력 채널	CH1, CH2
카운터 모드	Down counter
반복 모드	Repeat
초기 Duty	50% (range = 0)
Duty 범위	50~95%

3-2. TIM5 버튼 3초 인지 타이머

항목	설정값
TIM5 기준 주파수	10 kHz
TIM5 ARR	3000 - 1
길게 누름 설정값	3000 ms
인터럽트	Update interrupt, IRQ 30

4. VARIABLE

4-1. 열거형

```
typedef enum
{
    MOTOR_STOP,
    MOTOR_CW,
    MOTOR_CCW
} MOTOR_STATE;

typedef enum
{
    BTN_NONE,
    BTN_SHORT_PRESSED,
    BTN_LONG_PRESSED,
    BTN_UART
} BUTTON_EVENT;
```

4-2. GLOBAL VARIABLE

이름	Type	초기값	역할	변경 시점
button_event	event_t	EVT_NONE	현재 이벤트 저장	BUTTON, UART 인터럽트 발생시 <code>check_event()</code> 에서 갱신
motor_state	motor_state_t	MOTOR_STOP	현재 모터 상태 저장	<code>drive_motor()</code> 에서 모터 상태 변경 시 갱신
new_motor_state	motor_state_t	MOTOR_STOP	새로운 모터 회전 저장	<code>update_motor_state()</code> 에서 이벤트 처리 시 갱신
motor_speed	int	5	현재 PWM Duty 레벨 (0~9)	<code>drive_motor()</code> 에서 모터 상태 변경 시 갱신
new_motor_speed	int	5	새로운 PWM Duty 레벨 (0~9)	<code>update_motor_state()</code> 에서 이벤트 처리 시 갱신
Key_Pressed	volatile int	0	버튼 인터럽트 (falling edge) 발생 여부	<code>EXTI15_10_IRQHandler()</code> 에서 1로 세팅 → <code>check_event()</code> 에서 읽은 직후 0으로 리셋
Key_released	volatile int	0	버튼 인터럽트 (rising edge) 발생 여부	<code>EXTI15_10_IRQHandler()</code> 에서 1로 세팅 → <code>check_event()</code> 에서 읽은 직후 0으로 리셋

이름	Type	초기값	역할	변경 시점
uart_flag	volatile unsigned char	0	UART 인터럽트 발생 여부	USART2_IRQHandler()에서 1로 세팅 → check_event()에서 읽은 직후 0으로 리셋
uart_data	volatile unsigned char	0	UART로 수신된 문자	USART2_IRQHandler()에서 수신 시 갱신
timer_flag	volatile uint8_t	0	타이머 인터럽트 발생 여부	TIM4_IRQHandler()에서 갱신

5. METHOD

5-1. 주요 METHOD 요약

함수	입력 값	출력 값	변경 변수	역할
check_event()	Key_Pressed, Key_Released, uart_flag, timer_flag	없음 (void)	button_event, 각 인터럽트 플래그	버튼 및 UART 이벤트 판별
update_motor_state()	button_event, motor_state, motor_speed, uart_data	없음 (void)	new_motor_state, new_motor_speed	이벤트를 목표 모터 상태로 변환
drive_motor()	new_motor_state, new_motor_speed	없음 (void)	motor_state, motor_speed	모터 방향과 PWM Duty를 하드웨어에 반영

5-2. 주요 METHOD 상세 동작

5-2-1. check_event()

버튼, UART 및 타이머 인터럽트 플래그를 확인하여 다음 이벤트를 생성한다.

- Key_Pressed가 설정되면 PC13 핀 상태를 확인하여 버튼의 falling edge를 판별한다.
- Key_Released가 설정되면 PC13 핀 상태를 확인하여 버튼의 rising edge를 판별한다.
- 버튼이 3초 전에 해제되면 EVT_BTN_SHORT를 생성한다.
- 버튼이 눌린 상태로 3초가 지나면 EVT_BTN_LONG을 생성한다.
- uart_flag가 설정되면 EVT_UART를 생성한다.
- 확인이 끝난 인터럽트 플래그는 0으로 초기화한다.

호출 시점: 메인 루프에서 반복 호출한다.

5-2-2. update_motor_state()

`button_event`에 따라 목표 모터 상태와 목표 속도를 결정한다.

- `EVT_BTN_SHORT`
 - `MOTOR_STOP` 상태이면 `MOTOR_CW`로 전환한다.
 - `MOTOR_CW`와 `MOTOR_CCW` 상태에서는 회전 방향을 전환한다.
- `EVT_BTN_LONG`
 - `new_motor_state`를 `MOTOR_STOP`으로 변경한다.
- `EVT_UART`
 - `'f'`: `new_motor_state`를 `MOTOR_CW`로 변경한다.
 - `'r'`: `new_motor_state`를 `MOTOR_CCW`로 변경한다.
 - `'s'`: `new_motor_state`를 `MOTOR_STOP`으로 변경한다.
 - `'0'~'9'`: `new_motor_speed`를 해당 Duty 단계로 변경한다.

호출 시점: 메인 루프에서 `button_event != EVT_NONE`일 때 호출한다.

5-2-3. `drive_motor()`

현재 상태와 목표 상태를 비교하여 변경된 값만 하드웨어에 반영한다.

- `new_motor_state`와 `motor_state`가 다르면 GPIO 방향을 설정한다.
- 회전 방향이 반대로 변경될 때는 모터를 정지시킨 후 대기시간을 적용한다.
- `new_motor_speed`와 `motor_speed`가 다르면 PWM Duty를 변경한다.
- 적용이 끝나면 `motor_state`와 `motor_speed`를 갱신한다.

호출 시점: 메인 루프에서 반복 호출한다.

5-3. ISR

ISR	발생 조건	변경 변수	처리 내용
<code>EXTI15_10_IRQHandler()</code>	PC13 버튼의 falling/rising edge	<code>btn_flag</code>	<code>btn_flag</code> 를 1로 설정
<code>USART2_IRQHandler()</code>	USART2 수신 인터럽트	<code>uart_data</code> , <code>uart_flag</code>	수신 문자를 저장하고 <code>uart_flag</code> 를 1로 설정
<code>TIM5_IRQHandler()</code>	TIM5 3000ms 경과		3초마다 timeout 발생